📄 Firmware Documentation: ESP-NOW Receiver (Relay Unit)

This firmware is designed for the **Receiver Unit** (the ESP inside the 3D-printed box). It operates in **ESP-NOW mode**, meaning it receives direct commands from the Gateway without needing a WiFi router or internet connection.

🛠️ Hardware Requirements

- **Board:** ESP32 Dev Module
- **Pinout:**
  - **GPIO 15:** Connected to the Relay Signal Pin (IN).
  - **VCC/GND:** Powering the relay.

📦 Libraries

No external libraries are required. `esp_now.h` and `WiFi.h` are built-in to the ESP32 Arduino Core.

🚀 Installation Guide (Arduino IDE)

1. **Open the Sketch:** Copy the code above into a new sketch in Arduino IDE.
2. **Select Board:** Go to `Tools` -> `Board` -> `ESP32 Arduino` -> Select your specific board (e.g., *DOIT ESP32 DEVKIT V1*).
3. **Upload:** Connect your ESP via USB and click the **Upload (➡️)** button.
4. ⚠️ **CRITICAL STEP - Get the MAC Address:**
   - Once the upload finishes, open the **Serial Monitor** (Top right corner or `Ctrl+Shift+M`).
   - Set the baud rate to **115200**.
   - Press the **RST (Reset)** button on your ESP board.
   - Look for the line that says: `DEVICE MAC ADDRESS: XX:XX:XX:XX:XX:XX`
   - **Copy this address.** You will need to paste this inside the *Transmitter/Gateway* code so it knows where to send the signal.

🧠 Code Explanation

- **`#define RELAY_PIN 15`**: Defines which pin controls the relay. Change this if you wired your relay to a different pin.

- `struct_message`: A data structure defining what information is sent. It contains a simple `bool state` (True = ON, False = OFF). *Note: This structure must match exactly in the Transmitter code.*
- `OnDataRecv`: This is the "Callback Function." It runs automatically whenever a message arrives wirelessly. It reads the message and switches the relay HIGH or LOW.
- `WiFi.mode(WIFI_STA)`: Sets the ESP to "Station Mode." Even though we aren't connecting to a router, ESP-NOW requires the WiFi radio to be active.

❓ Troubleshooting

- **Relay not clicking?** Check if you are using a 3.3V or 5V relay. Some 5V relays need a logic level converter or 5V VCC to trigger properly.
- **Not receiving data?** Ensure the MAC address copied into the Transmitter code matches the one displayed in this unit's Serial Monitor exactly.

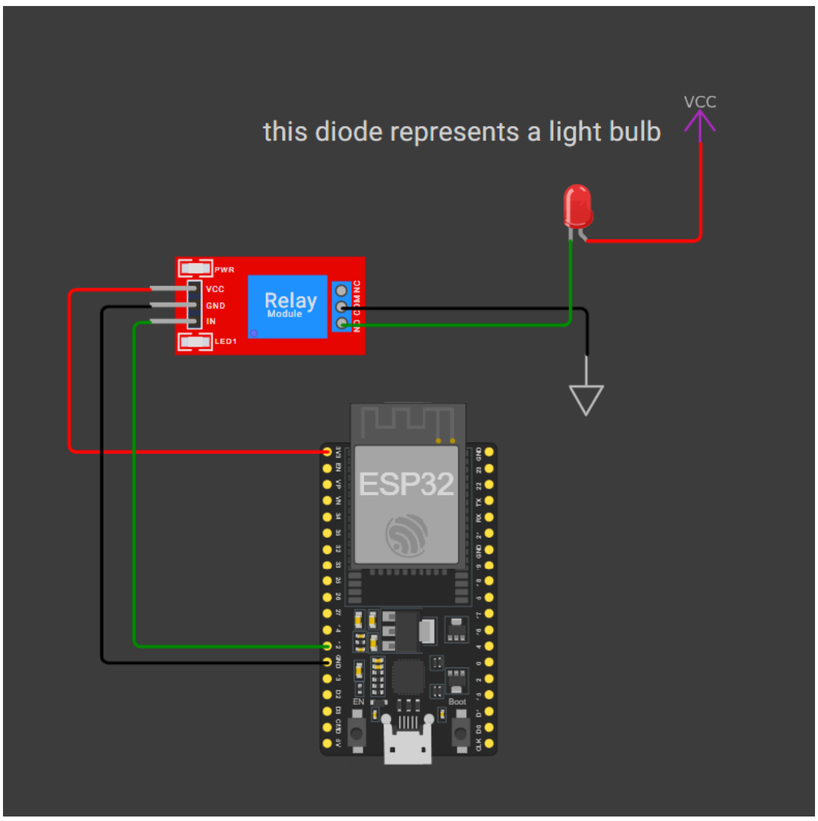
😴 Code

CODE :

```
/**
 * RECEIVER (Slave / Relay)
 * Controls the relay on the defined PIN based on ESP-NOW signal
 */
#include <esp_now.h>
#include <WiFi.h>

#define RELAY_PIN 15

// Data structure (must be identical in both ESPs)
typedef struct struct_message {
  bool state; // true = ON, false = OFF
} struct_message;

struct_message myData;

// Function called when data is received
void OnDataRecv(const uint8_t * mac, const uint8_t *incomingData, int len) {
  memcpy(&myData, incomingData, sizeof(myData));

  Serial.print("Command received: ");
  Serial.println(myData.state ? "ON" : "OFF");

  digitalWrite(RELAY_PIN, myData.state ? HIGH : LOW);
}

void setup() {
  Serial.begin(115200);
```

```
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, LOW);
  delay(1000);

  // Set WiFi to Station mode (required for ESP-NOW, but we don't connect to a router)
  WiFi.mode(WIFI_STA);
  delay(1000);

  // Print MAC Address (COPY THIS TO THE TRANSMITTER CODE!)
  Serial.println("---------------------------------------------");
  Serial.print("DEVICE MAC ADDRESS: ");
  Serial.println(WiFi.macAddress());
  Serial.println("---------------------------------------------");

  // Initialize ESP-NOW
  if (esp_now_init() != ESP_OK) {
    Serial.println("Error initializing ESP-NOW");
    return;
  }

  // Register receive callback function
  esp_now_register_recv_cb(esp_now_recv_cb_t(OnDataRecv));
}

void loop() {
  // Empty - everything happens in the interrupt (callback)
  delay(1000);
}
```

HOW TO CONNECT:



Code for relay testing:

```
#define RELAY_PIN 15

void setup() {
  pinMode(RELAY_PIN, OUTPUT);

  // startowo przekaźnik wyłączony
  digitalWrite(RELAY_PIN, HIGH);
}

void loop() {
  // WŁĄCZ przekaźnik
  digitalWrite(RELAY_PIN, LOW);
  delay(5000);

  // WYŁĄCZ przekaźnik
  digitalWrite(RELAY_PIN, HIGH);
  delay(5000);
}
```